

Inleiding:

In dit document staan code snippets van mijn project in DIPPER. Hieronder staan voorbeelden van code die ik zelf heb gemaakt binnen DIPPER binnen DI-Lab

Jinja2 MACRO (_config_fields.html)

Dit bestand bevat een Jinja macro, `render_config_fields`, die automatisch formulievelden opbouwt op basis van een lijst met configuraties. Per configuratie kiest het de juiste invoer (bijvoorbeeld tekst, getal, checkbox, datum, keuzelijst of bestand), vult bestaande waarden in bij bewerken en markeert verplichte velden.

Kort gezegd is dit bestand een macro die je in andere templates kunt hergebruiken, zodat je niet telkens dezelfde lange stuk code hoeft te kopiëren.

Als je bijvoorbeeld

```
{{ render_config_fields(configurations) }}
```

gebruikt, worden alle configuraties die je op de website hebt aangemaakt automatisch omgezet naar de juiste formulervelden, inclusief validatie en eventuele verplichte velden.

Frontend:

Backend: (regex filter gemaakt in JS)

[illegible]

```

document.addEventListener("DOMContentLoaded", function () {
    document.querySelector('form').addEventListener('submit', function (e) {
        if (!form.querySelector('.config-group')) return;

        form.addEventListener('submit', function (e) {
            form.querySelectorAll('.config-validation-error').forEach(n => n.remove());

            const groups = form.querySelectorAll('.config-group[data-required="1"]', '.config-group[data-regex="1"]');
            for (let g = 0; g < groups.length; g++) {
                const group = groups[g];
                const name = group.dataset.configname;
                const type = group.dataset.configtype;
                const regexPattern = group.dataset.configregex;
                const regenerator = group.dataset.configregenerator;
                let valid = true;
                let message = '';

                if (type === 'checkbox') {
                    const inputs = group.querySelectorAll('input[name="' + name + '"]');
                    const unchecked = Array.from(inputs).some(i => !i.checked);
                    if (!unchecked && group.dataset.required === "1") {
                        valid = false;
                        message = "Please select at least one option for \"" + name + "\"";
                    }
                } else if (type === 'array') {
                    const val = group.querySelector('select[name="' + name + '"]');
                    if (group.dataset.required === "1" && !val || !val.value) {
                        valid = false;
                        message = "Please select an option for \"" + name + "\"";
                    }
                } else if (type === 'file') {
                    const fileInput = group.querySelector('input[type="file"][name="' + name + '"]');
                    if (group.dataset.required === "1" && !fileInput || !fileInput.files.length) {
                        valid = false;
                        message = "Please upload a file for \"" + name + "\"";
                    }
                } else if (type === 'checkbox') {
                    const val = group.querySelector('input[type="checkbox"][name="' + name + '"]');
                    if (group.dataset.required === "1" && !val || !val.checked) {
                        valid = false;
                        message = "The checkbox \"" + name + "\" must be checked.";
                    }
                } else {
                    const input = group.querySelector('input[name="' + name + '"]');
                    const val = input ? input.value.trim() : '';
                    if (group.dataset.required === "1" && !val || !val.trim()) {
                        valid = false;
                        message = "The field \"" + name + "\" is required and cannot be empty.";
                    } else if (regexPattern && val) {
                        try {
                            const regex = new RegExp(regexPattern, "g");
                            if (!regex.test(val)) {
                                valid = false;
                                message = regex.test ? regenerator : "The field \"" + name + "\" does not match the required format.";
                            }
                        } catch (err) {
                            console.warn('Invalid regex for ' + name + ':', regexPattern);
                        }
                    }
                }
            }

            if (!valid) {
                e.preventDefault();
                const err = document.createElement('div');
                err.className = 'config-validation-error';
                err.style.color = 'red';
                err.style.marginBottom = '8px';
                err.textContent = message;
                group.parentNode.insertBefore(err, group);
                group.scrollIntoView({ behavior: 'smooth', block: 'center' });
                return;
            }
        });
    });
});
</script>
[% endmacro %]

```

PREPROCESSING DATA GRAFIEKEN JAVASCRIPT (*preprocess.js*)

Dit bestand bevat helperfuncties die bepalen hoe tekst moet worden voorbewerkt voordat statistieken worden berekend. Het leest welke optie de gebruiker kiest (geen splitsing, komma, spatie, streepje of punt), toont die keuze in de interface, en splitst daarna de invoer op basis van die keuze in losse onderdelen.

Kort gezegd is dit bestand een herbruikbare verwerkingslaag voor tekstvelden: één plek die regelt hoe invoer wordt opgeschoond en in tokens wordt verdeeld, zodat de rest van de statistiecode met nette data werkt.

Als je bijvoorbeeld de preprocess-keuze inlaadt via `getPreprocessMode` en daarna `splitStringTokens` gebruikt, wordt een waarde zoals “a, b, c” automatisch omgezet naar aparte items zonder extra spaties of lege stukjes.

```
JS preprocess.js X
DIPPER-main > app > static > js > stats > JS preprocess.js > ...
1 // This file handles preprocess options that split text into tokens.
2 // It helps normalize fields with commas, spaces, or other separators.
3 function getPreprocessMode(statId) {
4   const val = document.getElementById(`${statId}-preprocess`).value;
5   return val || 'none';
6 }
7
8 // Renders the preprocess UI block used by the stat builder.
9 function renderPreprocessUI(statId) {
10  return `
11    <div class="stat-row">
12      <div class="stat-col">
13        <label class="stat-label">Preprocess by:</label>
14        <select id="${statId}-preprocess" class="stat-select">
15          <option value="none" selected>None</option>
16          <option value="comma">,</option>
17          <option value="space">Space</option>
18          <option value="dash">-</option>
19          <option value="dot">.</option>
20        </select>
21      </div>
22    </div>
23  `;
24 }
25 // Stable stringify function to convert objects into a consistent string format for deduplication.
26 function splitStringTokens(raw, mode) {
27   if (raw === null || raw === undefined) return [];
28   const s = String(raw).trim();
29   if (!s) return [];
30   let parts = [];
31   switch (mode) {
32     case 'none':
33       parts = [s];
34       break;
35     case 'space':
36       parts = s.split(/\s+/);
37       break;
38     case 'dash':
39       parts = s.split('-');
40       break;
41     case 'dot':
42       parts = s.split('.');
43       break;
44     case 'comma':
45       parts = s.split(',');
46       break;
47   }
48   return parts.map((x) => String(x).trim()).filter(Boolean);
49 }
50
51
```

POPUP MET COLLECTIES BIJ GRAFIEKITEMS JAVASCRIPT (*collectionitemsmodal.js*)

Dit bestand zorgt voor een popup die laat zien welke collecties horen bij een onderdeel van een grafiek. Wanneer iemand op een balk of punt klikt, opent er een venster met de bijbehorende collecties. Als er geen collecties zijn, laat het bestand een lege melding zien.

Kort gezegd is dit een herbruikbaar onderdeel voor de statistiekenpagina: één plek die regelt hoe de popup wordt opgebouwd, geopend en gesloten, zodat de gebruiker snel kan zien welke collecties achter een grafiekpunt zitten.

Als je bijvoorbeeld op een grafiekelement klikt en daarna `openProjectsList` wordt uitgevoerd, haalt dit bestand de juiste collecties op uit de opgeslagen data en zet ze netjes in een lijst in de popup.

```
1 // This file manages the popup that lists collections per chart element.
2 // It shows which collections contributed to a bar or point.
3 function getOrCreateProjectsModal() {
4   let modal = document.getElementById('stats-projects-modal');
5   if (modal) return modal;
6
7   modal = document.createElement('div');
8   modal.id = 'stats-projects-modal';
9   modal.className = 'stats-projects-modal';
10  modal.innerHTML = `
11    <div class="stats-projects-modal__backdrop"></div>
12    <div class="stats-projects-modal__panel" role="dialog" aria-modal="true" aria-labelledby="stats-projects-modal-title">
13      <div class="stats-projects-modal__header">
14        <strong id="stats-projects-modal-title">Collections</strong>
15        <button type="button" class="stats-projects-modal__close">Close</button>
16      </div>
17      <div class="stats-projects-modal__body">
18        <div class="stats-projects-modal__empty muted">No collections found.</div>
19        <ul class="stats-projects-modal__list"></ul>
20      </div>
21    </div>
22  `;
23
24  document.body.appendChild(modal);
25
26  const close = modal.querySelector('.stats-projects-modal__close');
27  const backdrop = modal.querySelector('.stats-projects-modal__backdrop');
28  const hide = () => modal.classList.remove('is-open');
29  close?.addEventListener('click', hide);
30  backdrop?.addEventListener('click', hide);
31  document.addEventListener('keydown', (e) => {
32    if (e.key === 'Escape') hide();
33  });
34
35  return modal;
36 }
37
38 function openProjectsList(statId, key, label) {
39   const modal = getOrCreateProjectsModal();
40   const title = modal.querySelector('#stats-projects-modal-title');
41   const list = modal.querySelector('.stats-projects-modal__list');
42   const empty = modal.querySelector('.stats-projects-modal__empty');
43
44   const map = window[`sourcesByKey_${statId}`] || {};
45   const projects = Array.isArray(map[String(key)]) ? map[String(key)] : [];
46
47   if (title) title.textContent = label ? `Collections: ${label}` : 'Collections';
48   if (list) list.innerHTML = projects.map((p) => `<li>${escapeHtml(p)}</li>`).join('');
49   if (empty) empty.style.display = projects.length ? 'none' : 'block';
50
51   modal.classList.add('is-open');
52 }
53
```

OPGESLAGEN STATISTIEKEN TONEN EN VERWIJDEREN JAVASCRIPT (savedstats.js)

Dit bestand maakt kaarten aan voor opgeslagen statistieken en toont daar meteen de bijbehorende grafiek in. Het zorgt er ook voor dat een opgeslagen statistiek weer verwijderd kan worden, zowel uit de database als uit het scherm. Daarnaast houdt het rekening met een alleen-lezen pagina, waar sommige knoppen verborgen blijven.

Kort gezegd is dit een herbruikbaar onderdeel voor de statistiekenpagina: één plek die regelt hoe opgeslagen grafieken worden weergegeven, opgebouwd en verwijderd, zodat de gebruiker zijn opgeslagen statistieken makkelijk kan terugzien en beheren.

Als je bijvoorbeeld een opgeslagen statistiek toevoegt via `appendSavedStat`, maakt dit bestand automatisch een kaart met de naam van de statistiek, plaatst de grafiek op de juiste plek en koppelt eventueel een delete-knop eraan. Wanneer `deleteSavedStat` wordt gebruikt, haalt het de statistiek eerst weg via de server en verwijdert daarna de kaart uit de pagina.

```
function appendSavedStat(stat) {
  try {
    const container = document.getElementById('custom-stats-container');
    // Hide delete/type controls on read-only pages (the index.html page).
    const isReadOnly = container?.dataset?.statsView === 'readonly';
    const id = stat._id || `local-${Date.now()}`;
    const card = document.createElement('div');
    card.id = `saved-card-${id}`;
    card.className = 'saved-stat-card';
    // this is for the index page where we want to hide delete buttons and stat types, but still show the stat name and chart
    const metaHtml = isReadOnly
      ? ''
      : `
    <div class="saved-stat-card_meta">
      <small class="saved-stat-card_type">${stat.type} </small>
      <button class="saved-stat-card_delete" data-delete-saved="${id}">Delete</button>
    </div>
  `;
    // this is for the stats page where we want to show delete buttons and stat types, along with the stat name and chart
    card.innerHTML = `
    <div class="saved-stat-card_header">
      <strong>${stat.name} </strong>
    </div>
    <div class="saved-stat-card_canvas-wrap">
      <canvas id="saved-canvas-${id}" class="saved-stat-card_canvas"></canvas>
    </div>
  `;
    container.insertAdjacentElement('afterbegin', card);
    // this is for adding the event listener for delete button if not readonly
    if (!isReadOnly) {
      card.querySelector('[data-delete-saved]').addEventListener('click', () => deleteSavedStat(id));
    }

    const canvas = document.getElementById(`saved-canvas-${id}`);
    renderSavedStatFromQuery(stat, canvas, `saved-${id}`);
  } catch (e) {
    console.warn('appendSavedStat failed', e);
  }
}

// Removes a saved stat on the server and from the DOM.
async function deleteSavedStat(id) {
  try {
    const res = await fetch(`/api/saved-stats/${encodeURIComponent(id)}`, { method: 'DELETE' });
    if (res.ok) {
      const json = await res.json().catch(() => ({}));
      if (json.success === false) {
        console.warn('Delete returned failure', json);
      }
    } else {
      console.warn('Delete request failed with status', res.status);
    }
  } catch (e) {
    console.warn('deleteSavedStat failed', e);
  }

  try {
    const el = document.getElementById(`saved-card-${id}`);
    if (el) el.remove();
  } catch (e) {
    // ignore
  }
}
```

ID NAAR LABEL REFERENTIE OPZOEKINGEN VOOR GRAFIEKEN JAVASCRIPT (references.js)

Dit bestand helpt om technische ID waarden om te zetten naar leesbare namen voordat gegevens in grafieken worden getoond. Het controleert of een waarde eruitziet als een MongoDB ObjectId, bepaalt of een veld via een andere collectie moet worden opgezocht, en vraagt daarna de juiste labels op bij de backend.

Kort gezegd is dit een herbruikbare referentielaaag voor grafiekdata: één plek die ruwe ID's vertaalt naar begrijpelijke tekst, zodat de rest van de statistiekcde duidelijke labels kan tonen in plaats van databasecodes.

Als je eerst met `getRefSpec` bepaalt welke koppeling nodig is en daarna `fetchLabelMap` gebruikt, kunnen waarden zoals gekoppelde onderzoeker of cursus ID's automatisch worden vervangen door leesbare namen. Daardoor worden grafieken voor gebruikers veel duidelijker.

```
// This file helps turn ID fields into readable names.
// It looks up other collections so labels are correct in charts.
function isMongoObjectIdString(v) {
  if (v === null || v === undefined) return false;
  const s = String(v).trim();
  return /^[0-9a-fA-F]{24}$/.test(s);
}

// Maps a field to a lookup collection/label when IDs should be resolved.
function getRefSpec(mainCollection, field) {
  const c = String(mainCollection || '').trim();
  const f = String(field || '').trim();
  if (!c || !f) return null;

  const mapping = {
    projects: {
      course: { collection: 'course', labelField: 'name' },
      researcher: { collection: 'researchers', labelField: 'name' },
      research_project: { collection: 'research_projects', labelField: 'research_project' },
    },
    research_projects: {
      researcher: { collection: 'researchers', labelField: 'name' },
    },
  };

  return (mapping[c] && mapping[c][f]) ? mapping[c][f] : null;
}

// Fetches a map of id -> label from the backend.
async function fetchLabelMap(targetCollection, ids, labelField) {
  const uniq = Array.from(new Set((ids || []).map((x) => String(x)).filter(Boolean)));
  if (uniq.length) return {};

  const q = `query($collection:String!, $ids:[String!]!, $labelField:String){ lookupLabels(collection:$collection, ids:$ids, labelField:$labelField) }`;
  try {
    const res = await fetch('/graphql', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ query: q, variables: { collection: targetCollection, ids: uniq, labelField } }),
    });
    const jr = await res.json();
    const rows = (jr && jr.data && jr.data.lookupLabels) ? jr.data.lookupLabels : [];
    const out = {};
    (rows || []).forEach((r) => {
      if (!r || r.id === undefined || r.label === undefined) return;
      out[String(r.id)] = String(r.label);
    });
    return out;
  } catch (e) {
    return {};
  }
}
```

GRAPHQL VRAGEN EN DATABASEKOPPELING PYTHON (schema.py)

Dit bestand bevat de GraphQL-laag die bepaalt hoe gegevens uit de database kunnen worden opgevraagd door de applicatie. Het beschrijft welke soorten queries beschikbaar zijn, zoals het ophalen van collecties, voorbeelddata, gefilterde documenten, unieke waarden en configuraties, en koppelt die vragen aan functies die de juiste gegevens uit MongoDB halen.

Kort gezegd is dit bestand een herbruikbare tussenlaag tussen de interface en de database: één plek die regelt welke data mag worden opgevraagd, hoe die veilig wordt opgehaald, en hoe die wordt omgezet naar een vorm die de rest van de applicatie kan gebruiken.

Als je bijvoorbeeld via query documents documenten opvraagt of via lookup labels id's omzet naar leesbare namen, zorgt dit bestand ervoor dat de juiste databasequery wordt uitgevoerd en dat het resultaat netjes terugkomt als bruikbare data voor de website.

```
1 import sys
2 import os
3 import json
4 import logging
5 import graphene
6 from graphene.types.generic import GenericScalar
7 from bson.errors import InvalidId
8 from bson.errors import InvalidId
9 from pymongo.errors import PyMongoError
10 from app.db import db
11
12 def to_bson(doc):
13     """Convert a Mongo document into something we can safely JSON-encode"""
14     def sanitize_value(val):
15         if isinstance(val, (list, dict)) and value.__name__ == "dict":
16             return sanitize_value(val)
17         if isinstance(val, (bytes, bytearray)):
18             try:
19                 return value.decode("utf-8")
20             except UnicodeDecodeError as e:
21                 # Don't let bytes get passed in some distrits; bouded bytes
22                 # e values, transport-safe without dropping data.
23                 return b"..."
24             return base64.b64encode(bytes(value)).decode("ascii")
25         if isinstance(val, dict):
26             return {key: sanitize_value(val) for key, val in value.items()}
27         if isinstance(val, list):
28             return [sanitize_value(item) for item in value]
29         return value
30     if doc is None:
31         return None
32     if doc is None:
33         return None
34     out = {}
35     for key, value in doc.items():
36         if key == "_id":
37             out["id"] = sanitize_value(value)
38         else:
39             out[key] = sanitize_value(value)
40     return out
41
42 def resolve_collection(self, info):
43     if not info:
44         return None
45     return db.get_collection(name)
46
47 def resolve_collection_names(self, info, collection, limit=10):
48     if not info:
49         return None
50     coll = db[collection]
51     docs = coll.find()
52     return [doc['_id'] for doc in docs]
53
54 def resolve_query_documents(self, info, collection, filter=None, projection=None, limit=10):
55     if not info:
56         return None
57     coll = db[collection]
58     docs = coll.find(filter=filter, projection=projection, limit=limit)
59     return [doc['_id'] for doc in docs]
60
61 def resolve_distinct(self, info, collection, field):
62     if not info:
63         return None
64     coll = db[collection]
65     docs = coll.distinct(field)
66     return docs
67
68 class Query(graphene.ObjectType):
69     """Flexible GraphQL layer over MongoDB"""
70     collection = graphene.List(graphene.String)
71     collection_names = graphene.List(graphene.String)
72     query_documents = graphene.List(GenericScalar, collection=graphene.String(required=True), filter=graphene.String, projection=graphene.String, limit=graphene.Int)
73     distinct = graphene.List(GenericScalar, collection=graphene.String(required=True), field=graphene.String(required=True))
74
75     configurations = graphene.List(GenericScalar, collection=graphene.String(required=True), limit=graphene.Int)
76     lookup_labels = graphene.List(GenericScalar, collection=graphene.String(required=True), id=graphene.String(required=True), limit=graphene.Int)
77
78 def resolve_collection(self, info):
79     if not info:
80         return None
81     return db.get_collection(name)
82
83 def resolve_collection_names(self, info, collection, limit=10):
84     if not info:
85         return None
86     coll = db[collection]
87     docs = coll.find()
88     return [doc['_id'] for doc in docs]
89
90 def resolve_query_documents(self, info, collection, filter=None, projection=None, limit=10):
91     if not info:
92         return None
93     coll = db[collection]
94     docs = coll.find(filter=filter, projection=projection, limit=limit)
95     return [doc['_id'] for doc in docs]
96
97 def resolve_distinct(self, info, collection, field):
98     if not info:
99         return None
100     coll = db[collection]
101     docs = coll.distinct(field)
102     return docs
```